

Senior Fullstack Engineer take-home exercise

Why we're doing this

We want to understand how you design and build across the full stack — API design, data modelling, and a frontend that connects them thoughtfully. We care about trade-offs and reasoning, not polish. You may use AI tools. We'll ask you to explain and modify your solution during the interview.

What to build (open-ended on purpose)

Brighte would like to collect expressions of interest for a new product, **Brighte Eats**, and view those leads in a simple dashboard.

- Customers can express interest in one or more services.

Initial service types: `delivery`, `pick-up`, `payment`.

Note: Service types may change over time.

Core requirements

Backend

- **Relational database** (your choice; explain why)
- **GraphQL API** in **TypeScript** running on **Node.js**
- **1 mutation**
 - `register(name, email, mobile, postcode, services[])`
- **2 queries**
 - `leads(...)` — return a paginated list (limit/offset), filterable by service type, with sensible sorting
 - `lead(id)` — return a single lead and their service interests

Frontend

- **TypeScript** (any framework; explain your choice)
- **Registration form** — submits the `register` mutation; handles validation feedback, loading state, and success/error responses from the API
- **Leads dashboard** — displays the paginated leads list with filtering by service type; shows a lead detail view on selection
- The frontend and backend should be clearly integrated — the dashboard must work against your real API, not mocked data

General

- **Tests** — see below
- **README** — a must (see template below)

- **GitHub repo** — public or link-accessible; we prefer to see commits as you go
- We should be able to **check out and run** your solution easily

Non-functional expectations (choose and justify)

We're intentionally not prescribing all details. Make sensible decisions and explain them.

- **Data modeling:** How do you represent changing service types and multiple interests? (enum vs join table vs JSON; pros/cons)
- **Correctness & idempotency:** How do you avoid duplicate leads? (e.g., unique constraints, conflict handling)
- **Input validation:** Where does validation live — client, server, or both? Why?
- **Basic security:** avoid obvious injection, no secrets in git
- **Performance hygiene:** Avoid GraphQL N+1 where trivial; basic indexing; pagination approach
- **Frontend state & error handling:** How does the UI behave when the API is slow or returns an error?
- **Migrations:** How you evolve the schema, even in simple form

There is no single correct answer — we care most about your reasoning.

Optional stretch goals (self-select one if time allows)

Pick **one** if you have time and it strengthens your narrative. Breadth is not the goal here.

- Basic auth or admin boundary for the dashboard
- Audit trail or "change history" for service interests
- Rate-limiting on `register`
- Optimistic UI updates on the registration form

Testing

Include a small number of meaningful tests — we're not looking for exhaustive coverage, just evidence that you know what's worth protecting. Good candidates to test:

- A required field becoming optional (e.g. email or mobile)
- A new service type being added without the validation knowing about it
- The `register` mutation returning a lead on the happy path
- The registration form showing an error when the API returns a failure

Pick the tests that give you the most confidence for the least effort. A few well-chosen tests beat a large suite of shallow ones.

Deliverables checklist

- **GitHub repository** with incremental commits
- **README** (see template below)
- **Run instructions** (e.g., `docker-compose up`, `.env.example`)
- **Database migrations** or a bootstrap script
- **A small test suite** as described above
- **Design notes** on trade-offs you made and what you'd change at 10× scale

Timebox (guideline)

Aim for **2–2.5 hours** of focused work — AI tools are fair game and encouraged. A working end-to-end flow with a few well-chosen tests beats broad coverage with shallow thinking. It's fine to leave TODOs; just call them out. **We value clarity and ownership over volume.**

AI usage policy

You may use AI tools (Copilot, ChatGPT, Cursor, etc.).

Please add a short **"AI Assistance"** section in your README covering:

- Where AI helped (e.g., scaffolding components, writing tests)
- Places you verified or changed the AI's output
- Any notable limitations you encountered

We're assessing your **understanding and ownership**, not whether you typed every line.

README template

```
1 ## How to run
2 ## Why I chose [database / framework / frontend library]
3 ## Data modelling trade-offs
4 ## Validation strategy – client vs server
5 ## Idempotency approach
6 ## What I'd change at 10× scale
7 ## TODOs / known gaps
8 ## AI Assistance
```

Evaluation (what we look for)

- **Clarity of thought & ownership**
- **Data modeling** choices (and ability to evolve schema)
- **GraphQL/API design** (pagination, N+1 awareness)
- **Correctness & idempotency**
- **Frontend integration** — how the UI handles the API contract, loading states, and errors
- **Validation strategy** — where it lives and why
- **Test quality** (well-chosen, not exhaustive)
- **Change-readiness** (how easily can you adapt it live)

- **Communication** (explain trade-offs plainly)
-

Submission

Reply with a link to your repo and instructions to run locally. If you used a hosted DB or services, include sensible defaults and an `.env.example`. If anything is hard to run, explain what you'd do to simplify it.